A Learned Fixed-Size State for Long-Horizon Computer-Use Agents

Pengfei Ma

1 Problem and Current Practice

The performance bottleneck of computer-use agents (CUAs) has shifted from interface perception to long-horizon state maintenance. On OSWorld 2.0, the strongest current system completes only 20.6% of tasks within a 500-step budget [1]. The benchmark attributes failure to five dimensions, among which *long-horizon state drift* denotes agents that "keep working but do not converge"; two further dimensions, *stale grounding* and *stale click*, denote respectively a failure to update task state when new information arrives (Task 035) and actions applied to an outdated interface state (Task 052) [1]. In the benchmark's exposure-attribution analysis, 74% of trajectories that reach the Implicit-state Inference phenomenon fail because of it [1]. Related failures appear in other work: OpenCUA's error analysis describes three of its categories as "an incorrect action has no observable effect," "termination misjudgment," and "insert text one character off yet judge the action correct" [2]. Qwen-CUA reaches 86.2 on OSWorld-Verified but only 18.5 / 48.4 on OSWorld 2.0 [3]. The ICLR computer-use survey likewise lists long-horizon problems among the field's five bottlenecks [6].

These problems share a common source in the state mechanism of current systems. OpenCUA fixes visual history at three screenshots, retaining earlier information only as action text and supplementing it with a hand-written memory component [2]. Qwen-CUA expands the active history to 20 screenshots and folds the remainder by a deterministic rule, replacing folded screenshots with fixed text placeholders; the paper states that the procedure is "deterministic and requires no additional summarization model" [3]. Both therefore represent state as a recent-observation window plus hand-designed compression, and neither compression rule contains learnable parameters. Sager et al. observe that existing history-simplification methods are either manually crafted or based on generic summarization models, *both lacking adaptability to environments*, and argue that **history simplification should be a learnable component of the agent** [4]. Wu and Yang place **state-aware execution** first in their staged research agenda, with belief tracking as the candidate method [5].

2 Approach

Long-horizon tasks impose two requirements on the decision state. First, constraints that appear early must remain valid for the duration of the task; systems that violate this requirement exhibit *long-horizon state drift*. Second, information that arrives mid-task must be able to revise the existing task understanding; systems that violate this requirement exhibit *stale grounding*. Current systems meet both requirements with a fixed window: observations inside the window are retained passively, observations outside it are replaced by text or placeholders, and no discrimination of whether the existing understanding should be updated takes place.

We instead maintain the state as a continuously evolving object, so that retention and update are carried by a single dynamical process. Let $s_t \in \mathbb{R}^d$ be the task state at step t and o_t the encoding of the current screenshot. The state evolves according to a neural memory ordinary differential equation (nmODE) [7]:

$$\dot{s}(\tau) = -s(\tau) + f\big(s(\tau) + g(o_t)\big), \tag{1}$$

where $g(o_t)$ projects the current screenshot into state space and f is a nonlinear mapping. The term $-s$ lets the state decay in the absence of new input. The argument of the write term is $s + g(o_t)$: the new observation is added to the existing state before both pass through f, so new information *modifies* rather than *replaces* the existing state. The equilibrium $s^* = f(s^* + g(o_t))$ gives the value at which the state settles under the current observation.

On the discrete step we solve this equation directly with Heun's method, which consists of a predictor and a corrector:

$$\tilde{s}_{t+1} = s_t + \Delta t_t\, F(s_t; o_t), \qquad s_{t+1} = s_t + \frac{\Delta t_t}{2}\Big(F(s_t; o_t) + F(\tilde{s}_{t+1}; o_t)\Big), \tag{2}$$

where $F(s; o) = -s + f(s + g(o))$ and Δt_t is a learnable time increment. Expanding the two equations gives a self-retention coefficient of $1 - \Delta t_t + \Delta t_t^2/2$, which decreases monotonically in Δt_t: for small Δt_t the state changes slowly and history is retained, whereas as Δt_t approaches 1 the state approaches a function of the current observation. Δt_t therefore determines the extent to which the state is rewritten by the current observation; it is conditioned on the observation, takes a per-dimension value, and is bounded above by design. This mechanism corresponds to the learnable history simplification proposed by Sager et al. [4].

Choice of integration scheme. Existing discrete implementations of nmODE use first-order Euler [8]. In the CUA setting the state is updated several hundred times along a single trajectory, and the truncation error introduced at each discretization accumulates; Euler's global error is first-order in the step size, which long trajectories amplify. A second-order scheme reduces the per-step error, but at the cost of evaluating the vector field at two states. The projection term $g(o_t)$ within the vector field is substantially more expensive than the remainder, so a direct implementation nearly doubles the per-step cost. This cost is avoidable: $g(o_t)$ depends only on the observation and remains fixed within a task step, so the predictor and the corrector can share a single projection, with only the state-dependent mapping evaluated twice. This shared-projection mechanism has been proposed and validated in our prior work [9]. We therefore prioritize validating the second-order scheme.

3 Strengths and Limitations

The approach turns the question of "how many frames to keep" into "what the state should contain": the state dimension does not grow with trajectory length, and the update rule becomes a learnable component rather than a hand-designed one, consistent both with Sager et al.'s argument for learnable history simplification and with the first item of Wu and Yang's agenda. Its boundaries are threefold.

First, it does not address hidden state. The GUI is an incomplete projection of the system state [5]; if a piece of information never appears on screen—an order identifier, for example—no state model can recover it, and such failures still require a verification layer.

Second, it provides neither verification nor calibrated uncertainty. The state above is a deterministic point estimate, whereas Wu and Yang's agenda takes calibrated uncertainty and a lower false-success rate as evidence of progress [5]. Extending the deterministic state to a probabilistic one (latent-variable ODE / Neural SDE) is a natural next step.

Third, the natural point of comparison is a first-order discrete recurrence such as Mamba or another selective state-space model, not an RNN. Selective SSMs also carry a content-dependent timescale, so the distinguishing element of this approach lies in the integration order and the second-order accuracy enabled by shared projection, rather than in a learnable step size as such.

References

[1] M. Yuan, Z. Zhou, X. Xiong, W. Wu, J. Sun, J. Song, K. Cui, B. Wang, H. Wu, Y. Li, D. Lu, H. Lu, Q. Zhen, X. Wang, J. Deng, Y. Yang, C. Chen, B. Zheng, A. Su, X. Yu, H. Zou, S. Agashe, X. H. Lu, M. Kaur, Z. Qi, V. S. Chen, F. Sala, D. Liu, J. Lin, Z. Yu, Y. Su, S. Reddy, X. E. Wang, P. Qi, T. Xie, and T. Yu. OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. *arXiv preprint* arXiv:2606.29537, 2026.

[2] X. Wang, B. Wang, D. Lu, J. Yang, T. Xie, J. Wang, J. Deng, X. Guo, Y. Xu, C. H. Wu, Z. Shen, Z. Li, R. Li, X. Li, J. Chen, B. Zheng, P. Li, F. Lei, R. Cao, Y. Fu, D. Shin, M. Shin, J. Hu, Y. Wang, J. Chen, Y. Ye, D. Zhang, D. Du, H. Hu, H. Chen, Z. Zhou, H. Yao, Z. Chen, Q. Gu, Y. Wang, H. Wang, D. Yang, V. Zhong, F. Sung, Y. Charles, Z. Yang, and T. Yu. OpenCUA: Open foundations for computer-use agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025. proceedings.neurips.cc | doi:10.52202/085713-4669

[3] D. Lu, S. Bai, T. Bai, S. Fan, C. Gao, J. Guan, F. Hu, M. Huang, X. Huang, Y. Jiang, Y. Jing, D. Kong, N. Li, D. Liu, S. Liu, Z. Liu, Q. Shen, B. Wang, J. Wang, C. Wu, R. Xie, T. Xie, Z. Xie, H. Xu, A. Yang, T. Yu, W. Yuan, X. Zhang, Z. Zhang, M. Zhu, Z. Zhu, et al. Qwen-CUA: Native computer use for (almost) everything. *arXiv preprint* arXiv:2608.02352, 2026.

[4] P. J. Sager, B. Meyer, P. Yan, R. von Wartburg-Kottler, L. Etaiwi, A. Enayati, G. Nobel, A. Abdulkadir, B. F. Grewe, and T. Stadelmann. A comprehensive survey of agents for computer use: Foundations, challenges, and future directions. *Journal of Artificial Intelligence Research*, 85:34, 2026. doi:10.1613/jair.1.19490

[5] Y. Wu and Y. Yang. A platform-aware survey of execution, verification, and risk of graphical user interface agents. *ENGINEERING Information Technology & Electronic Engineering*, 27(7):1–26, 2026. jzus.zju.edu.cn | IEEE Xplore

[6] K. Marino, M. F. Ishmam, and A. Marasović. Computer use survey — a visual survey of computer use agents. In *ICLR Blogposts*, 2026. iclr-blogposts.github.io/2026/blog/2026/web-agent/

[7] Z. Yi. nmODE: neural memory ordinary differential equation. *Artificial Intelligence Review*, 56(12):14403–14438, 2023. doi:10.1007/s10462-023-10496-2

[8] L. Yu, J. Wu, B. Gou, X. Min, L. Zhang, Z. Yi, and T. He. MobileODE: An extra lightweight network. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, pp. 120931–120956, 2025. proceedings.neurips.cc | doi:10.52202/085713-4034

[9] P. Ma, L. Yu, L. Zhang, Z. Yi, and T. He. GhostODE: A lightweight network optimized by ordinary differential equation. *Neurocomputing*, 2026. doi:10.1016/j.neucom.2026.134898